

Simplified Reproduction of YOLOv10

End-to-End Object Detection without NMS on Pascal VOC

Validation Project — Introduction to Deep Learning

Guillaume Rabeau

Enzo Natali

Jonathan Bouniol

Anna Spira

May 2026

Contents

1	Introduction	2
2	Method	2
2.1	YOLOv10 Architecture	2
2.2	Differences with the YOLOv8 Baseline	3
3	Reproduction Setup	3
3.1	Dataset and Preprocessing	3
3.2	Training Protocol	3
3.3	Evaluation Metrics	3
3.4	Simplifications Compared to the Original Paper	4
4	Results	4
4.1	Learning Curves	4
4.2	Final Model Comparison	4
4.3	Efficiency and Latency	5
4.4	Per-Class Analysis	5
4.5	Qualitative Examples	6
5	Ablation: End-to-End vs. Forced NMS	8
5.1	Motivation and Hypothesis	8
5.2	Protocol	8
5.3	Ablation Results	8
5.4	Detection Count Distribution	9
6	Critical Discussion	9
6.1	What Works	9
6.2	What Does Not Work	9
6.3	Factors Explaining the Differences with the Paper	10
6.4	Limitations of Our Work	10
7	Conclusion	10

1 Introduction

The YOLO family of detectors has been the dominant paradigm in real-time object detection for several years, thanks to an effective trade-off between computational cost and detection performance. However, all models from YOLOv1 through YOLOv9 rely on *Non-Maximum Suppression* (NMS) as a post-processing step to filter redundant predictions, which adds non-negligible latency and makes performance sensitive to NMS hyperparameters.

The **YOLOv10** paper [1], presented at NeurIPS 2024, proposes two main contributions to address this issue:

1. **Consistent dual label assignments:** training with a *one-to-many* head (rich supervision) and a *one-to-one* head (NMS-free inference), linked by a consistent matching metric.
2. **Efficiency-driven architectural design:** compact inverted blocks (CIB), rank-guided block allocation, partial self-attention (PSA), and large-kernel convolutions to improve the accuracy-efficiency trade-off.

Project objective. We reproduce the paper’s conclusions in a simplified setting: training the Nano variants of YOLOv10 and YOLOv8 (baseline) on a subset of Pascal VOC, with comparative evaluation and an ablation study on the end-to-end vs. NMS mechanism.

2 Method

2.1 YOLOv10 Architecture

YOLOv10 retains the classic backbone–neck–head architecture of the YOLO family but introduces several key modifications:

Dual label assignments. During training, two detection heads operate in parallel:

- The *one-to-many* (o2m) head assigns multiple positive predictions to each object, providing a rich supervision signal.
- The *one-to-one* (o2o) head assigns only one prediction per object, enabling direct inference without NMS.

Both branches share the same backbone and neck. At inference time, only the o2o head is used, eliminating all post-processing.

Consistent matching metric. To align the supervision of both heads, a unified metric is used:

$$m(\alpha, \beta) = s \cdot p^\alpha \cdot \text{IoU}(\hat{b}, b)^\beta \quad (1)$$

where p is the classification score, $\hat{b}$ and b are the predicted and ground-truth bounding boxes, s is the spatial prior, and α, β control the balance between semantic prediction and localization quality. The paper shows that using the same α, β for both heads improves assignment alignment.

Compact Inverted Block (CIB). The CIB replaces standard residual blocks in stages with low intrinsic rank. It uses *depthwise* convolutions for spatial mixing and 1×1 pointwise convolutions for channel mixing, significantly reducing the parameter count.

Partial Self-Attention (PSA). A partial self-attention module is added after the CIB layers: only a fraction of the channels passes through the multi-head attention mechanism, while the rest is simply concatenated. This captures global dependencies without the full quadratic cost.

2.2 Differences with the YOLOv8 Baseline

YOLOv8 uses a C2f architecture (Cross Stage Partial with two convolutions) and relies entirely on *one-to-many* training with NMS at inference. Table 1 summarizes the main differences.

Table 1: Architectural comparison between YOLOv8n and YOLOv10n.

Component	YOLOv8n	YOLOv10n
Parameters (M)	3.01	2.71
Post-processing	NMS	End-to-end (NMS-free)
Training assignment	One-to-many	Dual (o2m + o2o)
Basic block	C2f / Bottleneck	CIB (rank-guided)
Attention	—	Partial self-attention

3 Reproduction Setup

3.1 Dataset and Preprocessing

We use the **Pascal VOC** dataset (20 object classes) via the `VOC.yaml` configuration provided by Ultralytics. To comply with hardware constraints, we use only **10 %** of the training data (`fraction=0.1`), i.e., approximately 1,640 images out of the 16,400 available. Evaluation is performed on the full VOC 2007 test set.

Images are resized to 416×416 pixels (instead of 640×640 in the original paper) to speed up training. Standard augmentations are retained: mosaic, horizontal flip, HSV adjustments, and `erasing=0.4`.

3.2 Training Protocol

Both models are trained with identical hyperparameters to ensure a fair comparison:

Table 2: Training hyperparameters common to both models.

Parameter	Value
Epochs	40
Batch size	8
Image size	416×416
Optimizer	SGD ($\text{lr}_0=0.005$, momentum=0.937)
Scheduler	Cosine annealing ($\text{lr}_f=0.001$)
Weight decay	5×10^{-4}
Warmup	3 epochs
Close mosaic	Last 10 epochs
Pretrained	Yes (COCO)
Seed	42
Device	Apple MPS (Mac Mini M2)

3.3 Evaluation Metrics

We report the standard object detection metrics:

- **mAP@50**: mean average precision at an IoU threshold of 0.5.
- **mAP@50:95**: mean over IoU thresholds from 0.5 to 0.95 with a step of 0.05 — the primary metric, stricter and more representative of localization quality.

- **Precision** and **Recall** at the optimal confidence threshold.
- **Latency** (ms/image) measured locally on 100 images with warmup.

3.4 Simplifications Compared to the Original Paper

- **Dataset**: Pascal VOC (20 classes, $\sim 1.6\text{k}$ training images) instead of COCO (80 classes, 118k images).
- **Fraction**: only 10 % of the training data.
- **Resolution**: 416×416 instead of 640×640 .
- **Epochs**: 40 instead of 500.
- **Scale**: only the **Nano** variants (lightest models).
- **Hardware**: Apple MPS instead of NVIDIA T4/A100 GPUs.
- **AMP**: disabled (MPS incompatibility at the time of experiments).

4 Results

4.1 Learning Curves

Figure 1 shows the evolution of mAP@50:95 on the validation set throughout training. YOLOv8n converges faster and maintains a consistent advantage over YOLOv10n across all 40 epochs. From epoch 5 onward, the gap stabilizes at approximately 7–8 mAP@50:95 points.

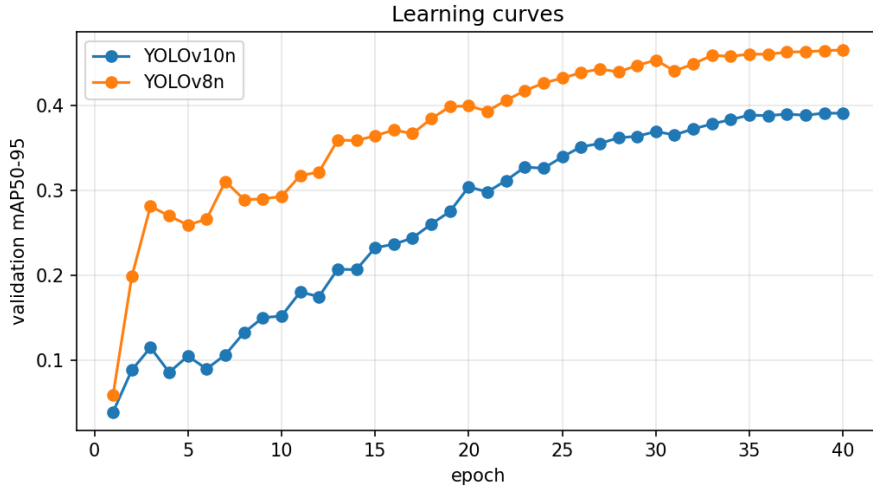


Figure 1: Learning curves — validation mAP@50:95 over epochs.

4.2 Final Model Comparison

Table 3 presents the evaluation results on the best checkpoint (`best.pt`) for each model.

Table 3: Final evaluation results on Pascal VOC test2007.

Model	mAP@50	mAP@50:95	Precision	Recall
YOLOv10n (end-to-end)	0.559	0.387	0.612	0.522
YOLOv8n (NMS)	0.680	0.466	0.725	0.621
Gap	−0.121	−0.079	−0.113	−0.099

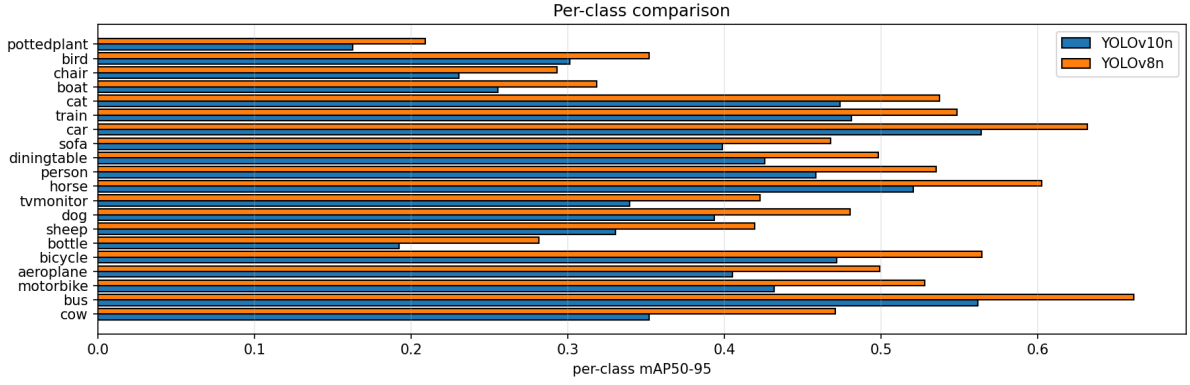


Figure 4: Per-class mAP@50:95 for YOLOv10n and YOLOv8n.

4.5 Qualitative Examples

Figure 5 shows visual predictions on validation images. YOLOv10n produces more parsimonious detections (fewer boxes per image), which is consistent with its end-to-end design, but misses certain objects that YOLOv8n detects correctly.

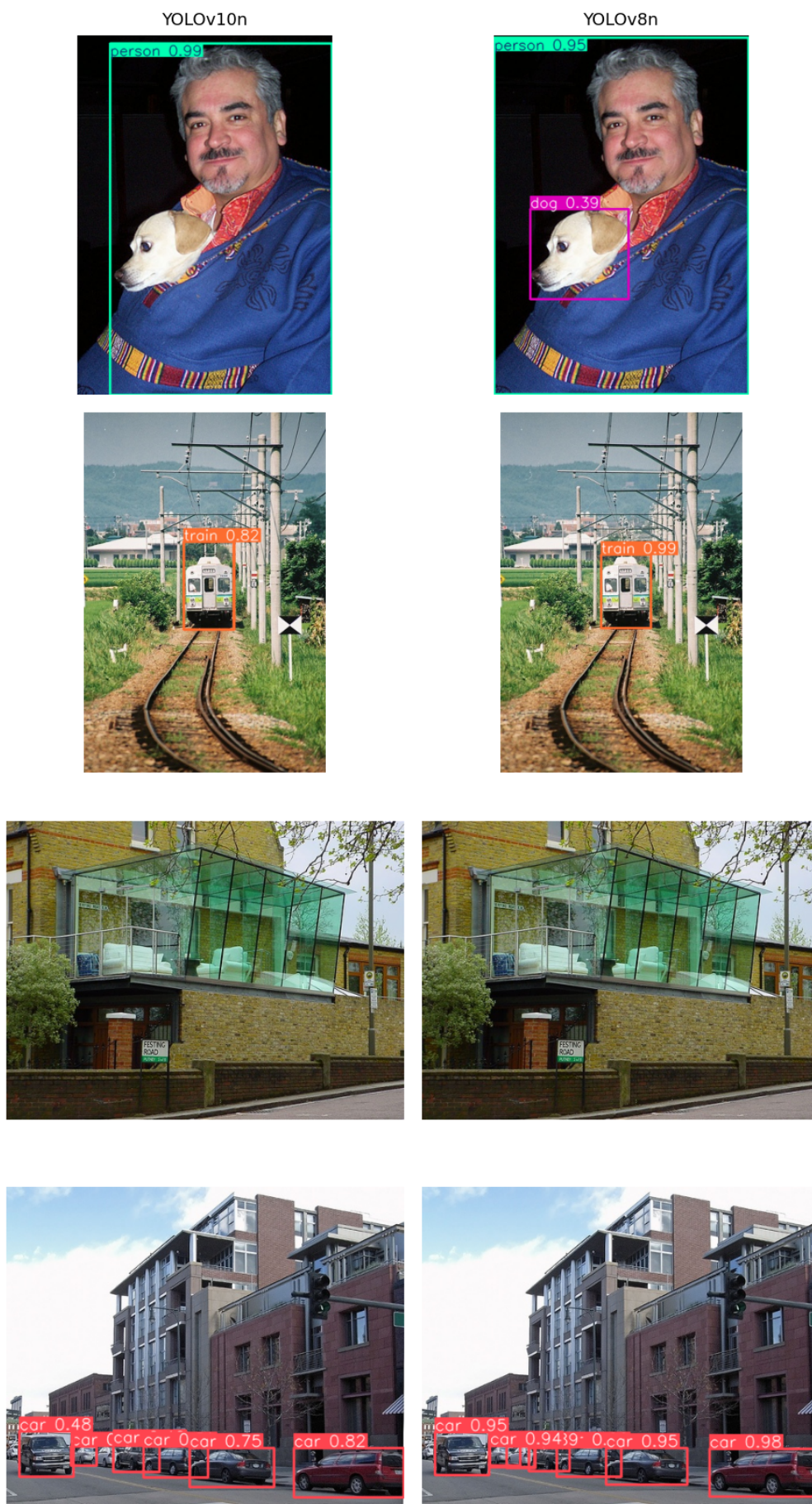


Figure 5: Detection examples on validation images (conf ≥ 0.25).

5 Ablation: End-to-End vs. Forced NMS

5.1 Motivation and Hypothesis

The main contribution of YOLOv10 is the elimination of NMS through the *one-to-one* head. We test the following hypothesis: *does end-to-end inference (without NMS) degrade performance compared to evaluation with forced NMS on the same checkpoint?*

If so, this would confirm the original paper’s observation: for small-capacity models (Nano), the o2o head fails to produce sufficiently discriminative features.

5.2 Protocol

We evaluate the *same* YOLOv10n checkpoint (`best.pt`) under three configurations:

1. **YOLOv10 end-to-end**: native NMS-free inference (o2o head only).
2. **YOLOv10 forced NMS**: NMS post-processing is forced on YOLOv10’s predictions.
3. **YOLOv8 (reference)**: baseline with standard NMS.

5.3 Ablation Results

Table 5: Ablation: end-to-end vs. forced NMS on the YOLOv10n checkpoint.

Configuration	mAP@50	mAP@50:95	Precision	Recall
YOLOv10 end-to-end	0.559	0.387	0.612	0.522
YOLOv10 forced NMS	0.665	0.452	0.697	0.618
YOLOv8 (reference)	0.680	0.466	0.725	0.621

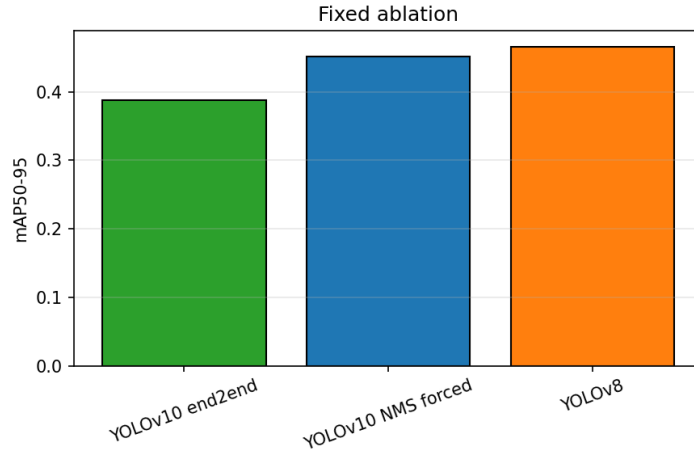


Figure 6: Ablation: mAP@50:95 by inference mode.

Analysis. Forcing NMS on the YOLOv10n checkpoint recovers **6.5 mAP@50:95 points** ($0.387 \rightarrow 0.452$), closing most of the gap with YOLOv8n (0.466). This demonstrates that:

1. The backbone and neck of YOLOv10n have learned representations of comparable quality to those of YOLOv8n.
2. The performance loss in end-to-end mode stems primarily from the *one-to-one* head, which is not sufficiently discriminative at this model scale.

3. This finding is **consistent with the original paper’s analysis** (Section 5, Table 3), which acknowledges a 1.0 % AP gap for YOLOv10-N on COCO between end-to-end and NMS modes.

The residual 1.4-point gap between YOLOv10 forced NMS and YOLOv8 can be explained by architectural differences (CIB vs. C2f) and the fact that our limited training conditions (40 epochs, 10 % of data) do not allow YOLOv10’s innovations to fully compensate.

5.4 Detection Count Distribution

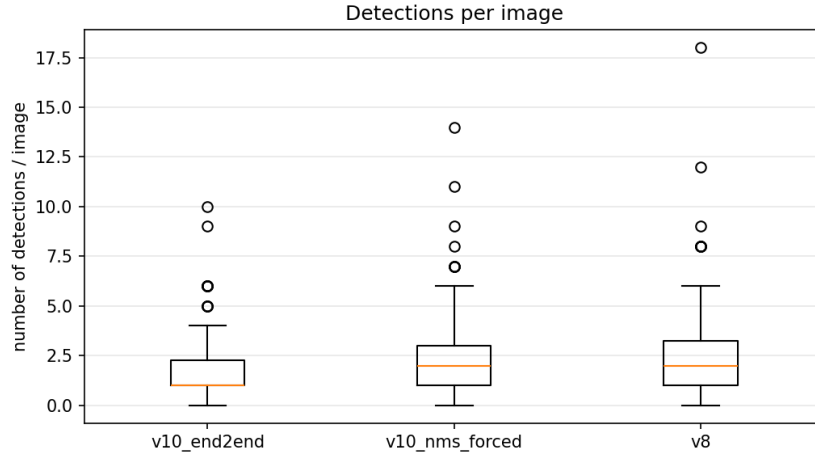


Figure 7: Distribution of detections per image by configuration.

Figure 7 confirms that end-to-end mode produces *fewer detections per image* (median ≈ 1 –2) than forced NMS (median ≈ 2 –3) or YOLOv8 (median ≈ 2 –3). This parsimonious behavior is expected by design: the o2o head selects at most one prediction per object, eliminating redundancies without post-processing.

6 Critical Discussion

6.1 What Works

- **The end-to-end vs. NMS ablation** qualitatively reproduces the paper’s observations: NMS-free mode incurs a performance loss for small models, and forced NMS recovers most of this gap.
- **The parsimonious behavior** of the o2o head is clearly visible in the detection count distribution: YOLOv10 end-to-end produces fewer boxes, consistent with its design.
- **The compactness** of YOLOv10n is confirmed: 10 % fewer parameters and 23 % fewer FLOPs than YOLOv8n.

6.2 What Does Not Work

- **YOLOv10’s accuracy advantage is not reproduced.** On COCO with 500 epochs, YOLOv10-N outperforms YOLOv8-N by 1.2 points. In our simplified setting, the reverse holds: YOLOv8n leads by 7.9 points. Several factors explain this gap.
- **The latency advantage is not observed.** The paper reports a latency of 1.84 ms (T4 TensorRT) for YOLOv10-N vs. 6.16 ms for YOLOv8-N. On MPS, YOLOv10n is slower because CIB and PSA operators do not benefit from Apple backend optimizations.

6.3 Factors Explaining the Differences with the Paper

Insufficient training duration. With only 40 epochs and 10 % of the data, the model sees approximately $40 \times 1,640 = 65,600$ images in total, compared to $500 \times 118,000 = 59,000,000$ in the original paper — a factor of $\sim 900\times$. YOLOv10’s architectural innovations (CIB, PSA, rank-guided design) likely require longer training to outperform the baseline.

COCO pretraining. Both models start from COCO-pretrained weights. This pretraining potentially advantages YOLOv8n because its C2f architecture is identical to the one used during pretraining, whereas YOLOv10n’s CIB blocks constitute a different architecture requiring more adaptation.

Reduced resolution. At 416×416 , YOLOv10n’s PSA module operates on smaller feature maps, reducing the benefit of partial self-attention compared to the 640×640 resolution used in the paper.

MPS backend. The Apple MPS backend is not optimized for the *depthwise* and partial attention operations used by YOLOv10, which nullifies the theoretical latency advantage.

6.4 Limitations of Our Work

- Training on a reduced fraction of Pascal VOC does not allow testing the hypothesis that YOLOv10 performs better at larger scale.
- We could not reproduce latency measurements on a T4 GPU with TensorRT, which is the paper’s reference setup.
- The study is limited to Nano variants; conclusions may differ for larger models (S, M, B) where the paper shows clearer gains.
- We did not explore the impact of the number of epochs on the relative convergence of the two models.

7 Conclusion

This project reproduced the main contributions of YOLOv10 in a simplified setting. The end-to-end vs. NMS ablation confirms that the *one-to-one* head effectively eliminates the need for post-processing, at the cost of a performance loss for small-capacity models — an observation consistent with the original paper. The architectural compactness of YOLOv10n (fewer parameters and FLOPs) is verified, but does not translate into a latency advantage on the MPS backend.

The overall performance gap in favor of YOLOv8n in our setting is primarily explained by the very limited training duration and small amount of data, which prevent YOLOv10’s innovations from being fully expressed. Complementary experiments with more epochs, higher resolution, and an NVIDIA GPU would be natural next steps.

References

- [1] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, G. Ding. YOLOv10: Real-Time End-to-End Object Detection. In *NeurIPS*, 2024. [arXiv:2405.14458](https://arxiv.org/abs/2405.14458).
- [2] G. Jocher. YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023.
- [3] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, 88(2):303–338, 2010.

- [4] J. Hosang, R. Benenson, B. Schiele. Learning Non-Maximum Suppression. In *CVPR*, 2017.
- [5] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko. End-to-End Object Detection with Transformers. In *ECCV*, 2020.
- [6] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, I.-H. Yeh. CSPNet: A New Backbone that can Enhance Learning Capability of CNN. In *CVPR Workshops*, 2020.
- [7] Y. Lv, Y. Xu, W. Ye, W. Zhao, J. Wei, C. Wang, J. Dang, S. Liu, B. Cao. DETRs Beat YOLOs on Real-time Object Detection. In *CVPR*, 2024.